

Lab 3: Architectural Justification

System: Centralized Audit Trail Compliance Engine (Problem Statement #50)

Name: Sanchita Sunil **SRN:** PES1UG24CS419

Architecture Selection

We chose **Microservices Architecture** for the Centralized Audit Trail Compliance Engine. The system is split into five independently deployable services: Log Ingestion, PII Masking, Immutable Audit Store, Query & Search, and Audit Export.

Style	Strength for this system	Weakness for this system
Layered	Clear separation of input, masking and storage logic	One deployable unit; ingestion and querying cannot be scaled separately
Client-Server	Central control and simple data consistency	Central server is a single point of failure; lost events break the audit trail
Microservices (chosen)	Independent scaling, fault isolation, PII handled by one service	More operational overhead and network calls between services

Reason 1: Ingestion and querying have very different loads

Enterprise applications send a continuous, bursty stream of JSON event logs, while compliance officers and auditors run heavy filtered searches across millions of historical records (NFR-001: 10 million records in under 1 second). As separate services, the Log Ingestion Service can scale out during logging peaks and the Query & Search Service can scale for audit season, without one slowing the other down.

Reason 2: Compliance logic changes on its own schedule, and must not stop logging

GDPR masking rules and export formats change over time. Because masking is its own service, new PII patterns (for example a new national ID format) can be added and redeployed without touching ingestion or querying. Fault isolation also matters here: if the Audit Export Service fails during a large export, the ingestion pipeline keeps recording events, and a gap in the audit trail would itself be a compliance failure.

Security Advantage: raw PII is confined to one service

Only the PII Masking Service ever handles raw Social Security and credit card numbers. The Immutable Audit Store accepts writes only through `IAuditWrite`, using a database role that can `INSERT` but cannot `UPDATE` or `DELETE`, and the Query and Export services connect through `IAuditRead` with read-only credentials. Each service gets least-privilege access, so compromising the query service exposes neither cleartext PII nor the ability to alter records. Every record carries a SHA-256 hash chained to the previous one, so any tampering is detectable in an export.

Performance Benefit: reads never compete with writes

The Query & Search Service is tuned only for reads: it keeps its own indexes on timestamp, source application and event type and can run several read replicas, while the ingestion path does fast append-only inserts. Heavy audit queries therefore do not slow down log ingestion, and the read path can be scaled until the under-1-second target for 10 million records is met.

Trade-off Accepted

Microservices add deployment and monitoring overhead and an extra network hop between ingestion and masking. We reduce this with gRPC and batched masking calls, and we keep a single append-only store as the source of truth to avoid consistency problems between services.